Version Management Guide

Purpose: Complete reference for all versioning policies, workflows, and procedures for both human contributors and AI agents.

Last Updated: 2026-01-24

---

1. Project Versioning Strategy

Semantic Versioning (SemVer 2.0.0)

- Follows standard SemVer format: MAJOR.MINOR.PATCH[-PRERELEASE]
- Alpha versions: `1.0.0-alpha.X`
- Beta versions: `1.0.0-beta.X`
- Release candidates: `1.0.0-rc.X`
- Releases: `1.0.0`, `1.1.0`, `2.0.0`

Version Source of Truth

- **Git Tags:** Authoritative for releases
- **`contest_tools/version.py`:** Contains `__version__` (should match most recent tag)
- **`README.md`:** Line 3 contains project version (must match `version.py`)
- Pre-commit hook automatically updates `__git_hash__` in `version.py`

Version Lifecycle

- **Alpha:** Early development, features may change
- **Beta:** Public testing, API stable
- **Release Candidate:** Pre-release verification
- **Release:** Production-ready, no suffix

---

2. Release Workflow

Overview

The complete release workflow is documented in `Docs/AI_AGENT_RULES.md` Section "Version Management" > "Release Workflow: Creating a New Version Tag". This section provides a summary and references the detailed workflow.

Key Steps (Summary)

1. Pre-Release Checklist
2. Create Draft Release Notes, Announcement, and CHANGELOG on Feature Branch; Commit

3. Merge Feature Branch into Master
4. Update Version References and Finalize Release Materials on Master (review/modify drafts if needed; see Section 3 for documentation updates)
5. Commit Version Updates (version bump - second commit on master)
6. Create and Push Tag
7. Post-Release Verification
8. Provide User Summary of VPS Update Steps (create ReleaseNotes/VPS_UPDATE_INSTRUCTIONS_* with four or five script commands for the VPS server)

For complete workflow details, see: `Docs/AI_AGENT_RULES.md` Section "Version Management" > "Release Workflow: Creating a New Version Tag"

---

3. Documentation Versioning Policy

CRITICAL: AI Agent Requirements

MANDATORY FOR AI AGENTS: This section defines binding rules for documentation version updates during releases.

Before performing any version updates, AI agents MUST:

1. Read `Docs/AI_AGENT_RULES.md` Section "Version Management"
2. Verify consistency between AI_AGENT_RULES.md and this document
3. If ANY discrepancy is detected: STOP immediately, report to user, wait for guidance
4. Only proceed if documents are consistent

See Section 5 for discrepancy detection procedures.

Category Definitions

Documentation is divided into five categories with different versioning rules:

Category A: User-Facing Documentation Files:

- `UsersGuide.md`
- `InstallationGuide.md`
- `ReportInterpretationGuide.md`

Versioning Rule: Match project version exactly

- Update version number on every project release
- Even if content doesn't change, version number must be updated
- This tells users "this documentation is current for version X"

Metadata Required:

```
**Version:** [Project Version, e.g., 1.0.0-alpha.10]
**Date:** YYYY-MM-DD
```

```
**Last Updated:** YYYY-MM-DD
**Category:** User Guide
```

Revision History:

- Always add entry for version bumps, even if no content changes
- Format: "Updated version to match project release vX.Y.Z (no content changes)"

Category B: Programmer Reference Files:

- `ProgrammersGuide.md`
- `PerformanceProfilingGuide.md`

Versioning Rule: Match project version exactly

- Same rules as Category A
- Update on every project release

Metadata Required:

```
**Version:** [Project Version, e.g., 1.0.0-alpha.10]
**Date:** YYYY-MM-DD
**Last Updated:** YYYY-MM-DD
**Category:** Programmer Reference
```

Revision History:

- Always add entry for version bumps, even if no content changes
- Format: "Updated version to match project release vX.Y.Z (no content changes)"

Category C: Algorithm Specifications Files:

- `CallsignLookupAlgorithm.md`
- `RunS&PAlgorithm.md`
- `WPXPrefixLookup.md`

Versioning Rule: Independent versioning (SemVer)

- Version increments only when algorithm logic changes
- Implementation-only changes do NOT increment algorithm version
- Use semantic versioning: MAJOR.MINOR.PATCH[-PRERELEASE]

Metadata Required:

```
**Version:** [Algorithm Version, e.g., 0.90.11-Beta]
**Date:** YYYY-MM-DD
**Last Updated:** YYYY-MM-DD
**Category:** Algorithm Spec
**Compatible with:** up to v[Current Project Version, e.g., 1.0.0-alpha.10]
```

"Compatible With" Field Management:

- Format: "up to vX.Y.Z" (range format, not list)
- Update the "up to" version with EVERY project release
- If algorithm changes (new version), reset "Compatible With" field:
  - Delete the "Compatible With" field entirely
  - Add note: "See previous version of this document (v[old-version]) for compatibility with earlier project versions"
  - Start new compatibility tracking from current project version
- If algorithm change is backwards compatible, keep existing "Compatible With" field and update "up to" version

When to Increment Algorithm Version:

- Algorithm logic changes
- Significant clarifications that affect understanding
- Breaking changes in behavior
- NOT: Minor wording improvements, formatting changes, implementation-only changes

Revision History:

- Add entry when algorithm version increments
- Add entry when "Compatible With" field is updated
- Document the reason for the change

Category D: Style Guides Files:

- `CLAReportsStyleGuide.md`

Versioning Rule: Independent versioning

- Version increments when style rules change
- Always "current" for developers (no "Compatible With" field needed)
- Developers should always use the latest version

Metadata Required:

```
**Version:** [Style Guide Version, e.g., 1.4.0]
**Date:** YYYY-MM-DD
**Last Updated:** YYYY-MM-DD
**Category:** Style Guide
```

No "Compatible With" Field: Style guides are always current.

Category E: In-App / Web-Embedded Help Artifacts:

- `web_app/analyzer/templates/analyzer/help_dashboard.html` - Dashboard navigation guide
- `web_app/analyzer/templates/analyzer/help_reports.html` - Report interpretation (Report List, Animation, reports)
- `web_app/analyzer/templates/analyzer/help_release_notes.html` - Release notes in-app view

- `web_app/analyzer/templates/analyzer/about.html` - About page (links to help, version)

Versioning Rule: No version field in templates. A **thorough check** is required before every release (during release prep on master). Update content when UI or features change so help stays accurate and complete.

Thorough Check (before each release):

1. **Coverage:** Every dashboard and help entry point (Help menu, About, in-dashboard help links) has corresponding, up-to-date content.
2. **Accuracy:** Each help template describes current behavior (e.g. Rate Differential pane, Report List drawer, Rate Chart, Band Activity, QSO/Points toggles). No references to removed or renamed features.
3. **Links and anchors:** In-app links (e.g. to release notes, other help pages) and any in-page anchors work.
4. **Optional:** Add a "Last updated" or release version in the help footer or About page if the project adopts it.

When: On the feature branch when adding or changing UI that affects help; and **during release prep on master** (Step 4: Update Version References and Finalize Release Materials) as a mandatory checkpoint before the version bump commit.

Standardized Metadata Format

All documentation files must include this metadata at the top:

```
**Version:** X.Y.Z[-suffix]
**Date:** YYYY-MM-DD
**Last Updated:** YYYY-MM-DD
**Category:** User Guide | Programmer Reference | Algorithm Spec | Style Guide
**Compatible with:** [Category C only] up to vX.Y.Z
```

Revision History Standards

Format:

```
### --- Revision History ---
## [Version] - YYYY-MM-DD
### Added | Changed | Fixed
- Description of change
```

Rules:

- Always add entry for version bumps, even if just version synchronization
- Document actual content changes separately from version bumps
- Use consistent format across all documents
- Reverse chronological order (newest first)

Example (Category A - Version Bump Only):

```
## [1.0.0-alpha.10] - 2026-01-24
### Changed
- Updated version to match project release v1.0.0-alpha.10 (no content changes)
```

Example (Category A - With Content Changes):

```
## [1.0.0-alpha.10] - 2026-01-24
### Changed
- Updated version to match project release v1.0.0-alpha.10
### Added
- Added Sweepstakes-specific features section
- Documented Enhanced Missed Multipliers report
```

Example (Category C - Compatible With Update):

```
## [0.90.11-Beta] - 2026-01-24
### Changed
- Updated "Compatible with" field to include project version v1.0.0-alpha.10
```

Documentation Index

Location: `Docs/ProgrammersGuide.md` (new section)

Purpose: Central reference listing all maintained documentation with current versions and categories.

Format:

```
## Documentation Index

**Purpose:** Central reference for all maintained documentation. See `Docs/VersionManagement

### User-Facing Documentation (Category A - Match Project Version)
- `UsersGuide.md` - Version: 1.0.0-alpha.10
- `InstallationGuide.md` - Version: 1.0.0-alpha.10
- `ReportInterpretationGuide.md` - Version: 1.0.0-alpha.10

### Programmer Reference (Category B - Match Project Version)
- `ProgrammersGuide.md` - Version: 1.0.0-alpha.10
- `PerformanceProfilingGuide.md` - Version: 1.0.0-alpha.10

### Algorithm Specifications (Category C - Independent Versioning)
- `CallsignLookupAlgorithm.md` - Version: 0.90.11-Beta (Compatible with: up to v1.0.0-alpha.
- `RunS&PAlgorithm.md` - Version: 0.47.4-Beta (Compatible with: up to v1.0.0-alpha.10)
- `WPXPrefixLookup.md` - Version: 0.70.0-Beta (Compatible with: up to v1.0.0-alpha.10)

### Style Guides (Category D - Independent Versioning)
- `CLAReportsStyleGuide.md` - Version: 1.4.0
```

For versioning policy details, see: `Docs/VersionManagement.md` Section 3

---

4. Maintenance Procedures

How to Update Versions

For Project Releases (Category A & B)

1. Update version number to match project version
2. Update "Last Updated" date
3. Add revision history entry
4. Verify metadata format is correct

For Algorithm Changes (Category C)

1. Determine if algorithm logic changed (not just implementation)
2. If changed: Increment algorithm version (SemVer)
3. Update "Compatible With" field:
   - If breaking change: Delete field, add note about previous version
   - If backwards compatible: Keep field, update "up to" version
4. Update "Last Updated" date
5. Add revision history entry

For Style Guide Changes (Category D)

1. Increment version when style rules change
2. Update "Last Updated" date
3. Add revision history entry

When to Update

Category A & B: Every project release (even if content unchanged) **Category C:** When algorithm changes OR every project release (for "Compatible With" field) **Category D:** Only when style rules change

Verification Steps

After updating versions:

1. Verify all Category A & B files match project version
2. Verify all Category C files have "Compatible With" field updated
3. Verify all metadata formats are consistent
4. Verify all revision history entries are added
5. Verify Documentation Index is updated

---

5. AI Agent Requirements

CRITICAL: Document Consistency Check

MANDATORY: Before performing any version updates, AI agents MUST:

1. **Read both documents:**
   - `Docs/AI_AGENT_RULES.md` Section "Version Management"
   - `Docs/VersionManagement.md` Section 3 (this section)
2. **Verify consistency:**
   - Check that category definitions match
   - Check that workflow steps align
   - Check that policy rules are consistent
   - Check that file lists match
   - Check that version number formats match
   - Check that update triggers match
3. **If ANY discrepancy is detected:**
   - **STOP immediately**
   - **DO NOT proceed with version updates**
   - **Report the discrepancy using the format below**
   - **Wait for explicit user guidance**

Discrepancy Detection Checklist

Before performing version updates, verify:

- ☐ Category definitions match (A, B, C, D)
- ☐ Category file lists match
- ☐ Workflow steps align (same steps, same order)
- ☐ Policy rules are consistent (when to update, how to update)
- ☐ "Compatible With" field rules match
- ☐ Revision history requirements match
- ☐ File locations match
- ☐ Version number formats match
- ☐ Update triggers match (when to update each category)

Discrepancy Reporting Format

If discrepancy detected, use this format:

```
**DISCREPANCY DETECTED - STOPPING WORKFLOW**

I detected a discrepancy between the versioning documents:

**Location:** [Section name in each document]
```

```
**Issue:** [Brief description]
**AI_AGENT_RULES.md says:** [Exact quote or summary]
**VersionManagement.md says:** [Exact quote or summary]

**Impact:** [What could go wrong if I proceed]

**Request for Guidance:**
1. Which document should I follow?
2. Should I update one document to match the other?
3. Is this an intentional difference, or a documentation error?

**Action:** Waiting for your guidance before proceeding with version updates.
```

What Constitutes a Discrepancy

- Different category definitions or lists
- Conflicting workflow steps or order
- Contradictory policy rules
- Different file lists to update
- Different version number formats
- Different update triggers
- Any other conflicting information

---

6. Quick Reference

Version Number Formats

- Project: `1.0.0-alpha.10`
- Algorithm: `0.90.11-Beta`
- Style Guide: `1.4.0`

Compatible With Format

- Range format: `up to v1.0.0-alpha.10`
- Updated with every project release
- Reset on breaking algorithm changes

Category Quick Reference

- **Category A:** User docs → Match project version
- **Category B:** Programmer reference → Match project version
- **Category C:** Algorithm specs → Independent versioning + "Compatible With"
- **Category D:** Style guides → Independent versioning, no "Compatible With"

File Locations

- Project version: `contest_tools/version.py`, `README.md` line 3
- Documentation: `Docs/*.md`
- Policy document: `Docs/VersionManagement.md` (this file)
- Workflow document: `Docs/AI_AGENT_RULES.md` Section "Version Management"

---

7. Examples

Example 1: Category A Update (Project Release)

File: `UsersGuide.md` **Current Version:** `1.0.0-alpha.4` **Project Version:** `1.0.0-alpha.10`

Changes:

1. Update version: `1.0.0-alpha.4` → `1.0.0-alpha.10`
2. Update "Last Updated": `2026-01-24`
3. Add revision history entry:
```
## [1.0.0-alpha.10] - 2026-01-24
### Changed
- Updated version to match project release v1.0.0-alpha.10 (no content changes)
```

Example 2: Category C Update (Compatible With Field)

File: `CallsignLookupAlgorithm.md` **Current Version:** `0.90.11-Beta` (unchanged) **Project Version:** `1.0.0-alpha.10`

Changes:

1. Keep algorithm version: `0.90.11-Beta`
2. Update "Compatible with": `up to v1.0.0-alpha.10`
3. Update "Last Updated": `2026-01-24`
4. Add revision history entry:
```
## [0.90.11-Beta] - 2026-01-24
### Changed
- Updated "Compatible with" field to include project version v1.0.0-alpha.10
```

Example 3: Category C Breaking Change

File: `RunS&PAlgorithm.md` **Current Version:** `0.47.4-Beta` **Algorithm Changed:** Yes (breaking change) **New Version:** `0.48.0-Beta`

Changes:

1. Update version: `0.47.4-Beta` → `0.48.0-Beta`
2. Delete "Compatible with" field

3. Add note: "See previous version of this document (v0.47.4-Beta) for compatibility with earlier project versions"
4. Update "Last Updated": `2026-01-24`
5. Add revision history entry:
```
## [0.48.0-Beta] - 2026-01-24
### Changed
- Algorithm logic updated (breaking change)
- See previous version (v0.47.4-Beta) for compatibility with earlier project versions
```

Related Documentation

- `Docs/AI_AGENT_RULES.md` - Release workflow and AI agent automation rules
- `Docs/Contributing.md` - SemVer overview and contribution guidelines
- `Docs/ProgrammersGuide.md` - Documentation Index and technical reference

Implementation Notes

This document was created as part of implementing a comprehensive versioning policy. It serves as the single source of truth for documentation versioning rules and procedures.

For AI Agents: This document contains binding rules that must be followed. Always verify consistency with `Docs/AI_AGENT_RULES.md` before proceeding.

For Human Contributors: This document provides complete guidance on how documentation versions are managed. When in doubt, refer to this document.